FedHydra: recap

Reviewed

Normalizzazione

Normalizzare le feature è utile perché in Hydra avremo kernel che vincono molte volte e avranno valori alti, mentre chi perde prende 0. Quindi abbiamo matrici sparse e se le andassimo a passare al classificatore Ridge come facevo inizialmente i valori assoluti molto grandi dominerebbero quindi dobbiamo normalizzare.

Primo andiamo a normalizzare a livello client prima di fare la Ridge per scegliere i top_k seeds:

```
x_trans = hydra_transform_seeds(x_train, client_seeds, n_ke
rnels,ts_length)

# Normalizzazione tramite SparseScaler
x_trans_normalized = SparseScaler.fit_transform(x_trans)

model = RidgeClassifierCV(alphas=np.logspace(-3, 3, 10)).fi
t(x_trans_normalized, y_train)
```

SparceScaler l'ho copiato dalla libreria e salvato su utils.py

Seconda Normalizzazione

NB: il codice inizialmente qui era composto da 1 unico grande ciclo su client_id in range(n_clients), che andava da hydra_transform_seeds alla creazione delle matrici A e B. Ora l'ho diviso in 3 cicli

Qua ho calcolato i primi pezzi per calcolare la dev standard.
Uso una lista_x_train per evitare di dover ricalcolare dopo x_trans

```
sum_columns = []
sum_squared = []
N_samples = []
```

```
list_x_trans = []
for client_id in range(n_clients):
    x_train, y_train = s_X_train[client_id], s_Y_train[clie
nt_id]
    # ora questi seed non sono più casuali, ma sono la list
a finale dei kernel migliori condivisa da tutti
    client_seeds = dict_seeds[client_id]

    x_trans = hydra_transform_seeds(x_train, client_seeds,n
_kernels, ts_length)
    list_x_trans.append(x_trans)

    sum_columns[client_id].append(np.sum(x_trans, axis=0))
    sum_squared[client_id].append(np.square(x_trans, axis =
0))
      N_samples[client_id].append(len(x_trans))
```

Dopo di che lato server: aggrego e calcolo

```
for client_id in range(n_clients):
    sum_global += sum_columns[client_id]
    sum_squared_global += sum_squared[client_id]
    N_samples_global += N_samples[client_id]

varianza_global = (sum_squared_global / N_samples_global) -
np.square(sum_global / N_samples_global)

dev_standard_global = np.sqrt(varianza_global) + 1e-10
```

Poi lato client: scalo ogni colonna di x_trans per dev_standard_global e poi calcolo A e B local

```
A_global = None
b_global = None
A_locals, b_locals = [], []

for client_id in range(n_clients):
    x_trans = list_x_trans[client_id]
```

```
    y_train = s_Y_train[client_id]

    x_trans_scaled = x_trans / dev_standard_global
    # Converte le etichette (es. la classe 2) in formato "O
ne-Hot" (es. [0, 0, 1]
    y_onehot = np.eye(n_classes)[y_train]
    # Impossibile fare il processo inverso
    A_local = x_trans_scaled.T @ x_trans_scaled  # Matrice
di covarianza delle feature
    b_local = x_trans_scaled.T @ y_onehot # Correlazione tr
a feature e classi (Z^T * Y)

    # Se è il primo client, capiamo la dimensione REALE del
le feature e creiamo le matrici globali
    if A_global is None:
        n_features_reali = A_local.shape[0]
        A_global = np.zeros((n_features_reali, n_features_r
eali))
        b_global = np.zeros((n_features_reali, n_classes))

    A_locals.append(A_local)
    b_locals.append(b_local)

    A_global += A_local
    b_global += b_local
```